

PORTFOLIO PROJECT REPORT

AI-Powered E-commerce Sales Analysis & Automated Reporting

An n8n workflow combining Google Sheets, JavaScript analytics, Claude AI, GitHub and Gmail

Item	Detail
Prepared by	Panicker Nimesh Mahendran
Role	Data Analyst (Fresher)
Report date	2 October 2026
Repository	github.com/nimeshpanicker/AI-Ecommerce-Sales-Reporting
Tools	n8n, JavaScript, Claude (Sonnet 5.5), Google Sheets, GitHub, Gmail

Table of Contents

- 1. Executive Summary 3
- 2. Project Overview 3
- 3. Workflow Architecture 4
- 4. Pipeline A: Conversational Analyst 5
- 5. Pipeline B: Automated Weekly Report 6
- 6. Data Processing Layer 6
- 7. AI Layer and Guardrails 7
- 8. Delivery: GitHub and Gmail..... 8
- 9. Sample Output and Results 8
- 10. Design Review and Improvements..... 11
- 11. Skills Demonstrated 12
- 12. Conclusion 12

1. Executive Summary

This project automates the full path from raw e-commerce transactions to a stakeholder-ready sales report. A single **n8n** workflow reads **9,994 transaction rows** from Google Sheets, computes KPIs and breakdowns in JavaScript, hands a compact summary to **Claude** for narrative analysis, then publishes the result to **GitHub** and delivers it by **Gmail**.

The workflow contains two independent pipelines that share the same data source and the same design principle: the language model never sees raw records, only pre-computed aggregates.

- **Pipeline A, Conversational Analyst:** a public chat interface where a user asks questions about the sales data and an AI agent with short-term memory answers using only the computed summary.
- **Pipeline B, Automated Weekly Report:** a schedule-triggered flow, every Monday at 09:00, that generates a structured Markdown report, commits it to the repository under a date-stamped file name and emails the same report.

Headline results in the sample report

Metric	Value
Total revenue	2,297,200.86
Total profit	286,397.02
Overall profit margin	12.47%
Distinct orders / customers	5,009 / 793
Strongest category	Technology (836,154.03 revenue, 145,454.95 profit)
Weakest-margin category	Furniture (2.49% margin)
Strongest region	West (725,457.82 revenue)

Why this design matters

Separating deterministic computation (JavaScript) from interpretation (Claude) keeps every number traceable to code while still delivering readable business commentary. Strict prompts forbid invented or estimated values.

2. Project Overview

Business problem

Sales reporting is usually repetitive: export data, rebuild pivots, write commentary, share. It is slow, inconsistent between weeks and dependent on one person. This project removes the manual steps while keeping the analysis grounded in verifiable calculations.

Objectives

- Calculate core KPIs (revenue, profit, margin, orders, customers, average order value, quantity) automatically from the live sheet.
- Break performance down by product, category, region and month.
- Check basic data quality (duplicates, missing values) on every run.

- Generate a consistent, professional report with an AI model without allowing fabricated figures.
- Publish the report as a portfolio artifact and email it to stakeholders.
- Allow ad-hoc questions through a chat interface using the same computed summary.

Technology stack

Component	Purpose
n8n	Workflow orchestration, triggers, node chaining
Google Sheets (OAuth2)	Source dataset, "Ecommerce Sales Analysis" workbook, "Data" tab
JavaScript (n8n Code nodes)	Column detection, KPI maths, grouping, monthly trend, data-quality checks
Claude Sonnet 5.5 (Anthropic API)	Narrative analysis, insights, recommendations, chat answers
n8n AI Agent + Window Buffer Memory	Conversational layer with a 10-message context window
GitHub (OAuth2)	Versioned storage of generated reports under /reports
Gmail (OAuth2)	Delivery of the report by email

3. Workflow Architecture

The canvas contains two chains, visualised below. The top chain is on-demand; the bottom chain is scheduled.



Figure 1. Logical architecture of the two pipelines



Figure 2. The workflow as built in n8n (the scheduled run shows 9,994 items reduced to 1 summary item)

Node inventory

Node	Type	Role
When chat message received	Chat Trigger	Public chat UI titled "Your First Insight"; replies from the last node
Get row(s) in sheet	Google Sheets	Reads all rows of the Data tab for Pipeline A
Code in JavaScript	Code	Builds the compact analytical summary and attaches the user question
AI Agent	AI Agent	Answers questions from the summary using a system prompt
Anthropic Chat Model	Claude model	Language model for the agent
Simple Memory	Window Buffer Memory	Keeps the last 10 messages of context
Schedule Trigger	Schedule	Weekly, Monday at 09:00
Get row(s) in sheet1	Google Sheets	Reads all rows for Pipeline B (9,994 items)
Reporting KPI & Analysis	Code	Produces KPI, product, category, region, monthly and quality summary (1 item)
Basic LLM Chain	Basic LLM Chain	Prompts Claude to write the structured report
Anthropic Chat Model1	Claude model	Language model for the chain
Create a file	GitHub	Commits reports/ecommerce_sales_report_YYYY-MM-DD.md
Send a message	Gmail	Emails the report as plain text

4. Pipeline A: Conversational Analyst

Pipeline A turns the dataset into something a non-technical user can interrogate in plain language.

Flow

1. A user opens the public chat and sends a question, for example "Which region has the lowest margin?".
2. The Google Sheets node reads the full Data tab.
3. The Code node computes the analytical summary and attaches the original question.
4. The AI Agent receives the question plus the JSON summary and answers using only those figures.
5. Simple Memory keeps the last 10 messages so follow-up questions work.

What the summary contains

- Dataset size and the list of detected columns
- KPIs: total revenue, profit, quantity, orders, customers, average order value, profit margin
- Top 10 products by revenue, plus category and region breakdowns
- Monthly revenue, profit and margin
- Data quality: duplicate row count and per-column missing values

Behaviour when data is missing

If a required column cannot be found, the corresponding KPI is returned as null rather than zero. The agent is instructed to say that the source column was unavailable instead of guessing, which keeps answers honest when the sheet structure changes.

Agent rules

- Never invent numbers or claim to have analysed raw records.
- Preserve ranking order exactly as calculated.
- Separate each insight into Observed result, Interpretation and Recommendation.
- When asked for a report, follow a fixed nine-section structure (Executive Summary through Business Recommendations).

5. Pipeline B: Automated Weekly Report

Pipeline B is the hands-off reporting engine. It needs no human input once activated.

Flow

1. Schedule Trigger fires every Monday at 09:00.
2. Google Sheets returns all 9,994 rows as separate items.
3. The Reporting KPI & Analysis node collapses them into a single JSON summary.
4. The Basic LLM Chain sends that summary to Claude with a detailed report prompt.
5. The report is committed to GitHub as a Markdown file named with the run date.
6. Gmail sends the same text to the stakeholder inbox.

Report structure requested from Claude

#	Section	Content
1	Executive Summary	Concise overview of performance
2	KPI Summary	Table of revenue, profit, margin, orders, customers, AOV, quantity
3	Product Performance	Top 10 products with revenue, profit, margin, quantity
4	Category Performance	Revenue, profit, quantity, margin by category
5	Regional Performance	Same measures by region, flagging divergent patterns
6	Monthly Performance	Trend and notable changes
7	Profitability Analysis	Margin, high-revenue/low-profit areas
8	Data Quality	Duplicates, missing values, dataset size
9-11	Insights, Recommendations, Conclusion	5 to 7 insights, practical actions, management conclusion

A second instruction block asks for a public-portfolio version: a fixed title, a Project Overview, and a closing Technical Workflow section describing the chain Google Sheets, Data Processing, KPI Analysis, Claude AI, Gmail, GitHub. It explicitly excludes customer names, emails, phone numbers, any personal data and raw transaction records.

6. Data Processing Layer

Both Code nodes follow the same logic. They are the analytical core of the project, and every figure in the final report originates here.

Column detection

Instead of hard-coding column names, the script searches the header row for common aliases, case-insensitively and ignoring surrounding spaces. This makes the workflow tolerant to renamed columns.

Field	Accepted aliases
Order ID	Order ID, OrderID, Order_Id, order_id
Order date	Order Date, OrderDate, Order_Date, Date
Customer	Customer ID, CustomerID, Customer_Id, customer_id
Product	Product, Product Name, ProductName, Product ID, ProductID
Category / Sub-category	Category, Product Category / Sub-Category, Sub Category, SubCategory
Region	Region, Sales Region
Quantity	Quantity, Qty
Sales	Sales, Revenue, Amount, Total Sales
Profit	Profit, Net Profit

Calculations

Measure	Method
Total revenue / profit / quantity	Sum of the detected Sales, Profit and Quantity columns after cleaning numbers (commas removed; the chat pipeline also strips the rupee sign and %)
Total orders	Count of distinct Order ID values (falls back to row count if absent)
Total customers	Count of distinct Customer ID values
Average order value	Total revenue divided by distinct orders
Profit margin	Total profit divided by total revenue, multiplied by 100
Group breakdowns	Revenue, profit and quantity summed per product, category or region; margin recomputed per group; sorted by revenue descending; products limited to the top 10
Monthly trend	Order Date parsed, bucketed as YYYY-MM, sorted chronologically, with margin per month
Data quality	Duplicate rows via full-row JSON signature; missing values counted per column

Differences between the two Code nodes

Aspect	Pipeline A (chat)	Pipeline B (scheduled)
Unavailable metric	null	0
Rounding	Raw values	Rounded to 2 decimals
Currency symbol handling	Strips the rupee sign	Not stripped
Extra fields	User question, record counts	Column names list
Empty dataset	Returns an error object	Returns an empty summary structure

7. AI Layer and Guardrails

Prompt design is the main safeguard against hallucination. Both Claude steps receive the same type of input: a compact JSON summary, never raw rows.

Guardrails used

- Analyse only the supplied summary.
- Do not invent, estimate or alter calculated values.
- Do not assign a currency unless the dataset specifies one.
- If a metric is null, state that the source column was unavailable.
- Keep observations separate from interpretation and recommendations.
- Keep the tone professional and suitable for business stakeholders.
- For the public artifact, exclude personally identifiable information and raw records.

Privacy by design

Only aggregates leave the Code node. Customer-level and transaction-level data are never included in the prompt, which reduces both data-exposure risk and token cost, and is why the same approach scales to the full 9,994-row dataset.

Two AI patterns in one project

	AI Agent (Pipeline A)	LLM Chain (Pipeline B)
Interaction	Multi-turn, interactive	Single call, fully automated
Memory	10-message window	None
Output	Answer to the user question	Structured Markdown report
Prompt style	System message plus question	Long template with fixed headings

8. Delivery: GitHub and Gmail

GitHub

The GitHub node creates a Markdown file in the repository using the pattern `reports/ecommerce_sales_report_YYYY-MM-DD.md` with the commit message "Add automated ecommerce sales report". Each run therefore leaves a dated, version-controlled report that can be browsed, diffed and shown in a portfolio.

Gmail

After the commit succeeds, the Gmail node sends the report text to a fixed recipient with the subject "E-commerce Sales Performance Report". Sending after the commit means the email is only delivered when the archive step has worked.

Repository layout

Path	Contents
/workflow	Exported n8n workflow JSON (AI-Ecommerce-Sales-Reporting.json)
/screenshots	Canvas screenshot of the workflow
/reports	Date-stamped reports committed by the workflow
README.md	Full e-commerce sales analysis report

9. Sample Output and Results

Source note

The figures in this section are taken from the analysis report published in the repository README, prepared from the full 22-column transaction dataset (9,994 records). The weekly files that the workflow writes to /reports were not reviewed for this document, and the README includes breakdowns (annual totals, distinct state and city counts) that go beyond the summary the workflow computes.

Overall statistics

Metric	Value
Records / columns	9,994 / 22
Distinct orders	5,009
Distinct customers	793
Distinct product IDs / names	1,862 / 1,841
Total revenue	2,297,200.86
Total profit	286,397.02
Profit margin	12.47%
Total quantity	37,873
Average order value	458.61
Order date range	2011-01-04 to 2014-12-31

Category performance

Category	Revenue	Profit	Margin	Quantity	Orders
Technology	836,154.03	145,454.95	17.40%	6,935	1,710
Furniture	741,999.80	18,451.27	2.49%	8,028	1,868
Office Supplies	719,047.03	122,490.80	17.04%	22,910	4,308

Regional performance

Region	Revenue	Profit	Margin	Quantity	Orders
West	725,457.82	108,418.45	14.94%	12,266	1,611
East	678,781.24	91,522.78	13.48%	10,618	1,401
Central	501,239.89	39,706.36	7.92%	8,780	1,175
South	391,721.91	46,749.43	11.93%	6,209	822

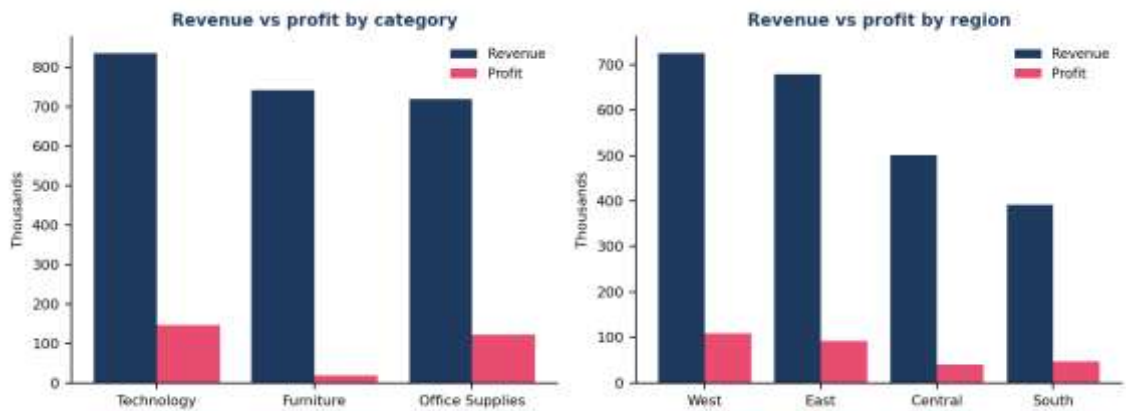


Figure 3. Revenue and profit by category and region (thousands)

Top products by revenue

Product	Revenue	Profit	Margin
Canon imageCLASS 2200 Advanced Copier	61,599.82	25,199.93	40.91%
Fellowes PB500 Electric Punch Plastic Comb Binding Machine	27,453.38	7,753.04	28.24%
Cisco TelePresence System EX90 Videoconferencing Unit	22,638.48	-1,811.08	-8.00%
HON 5400 Series Task Chairs for Big and Tall	21,870.58	0.00	0.00%
GBC DocuBind TL300 Electric Binding System	19,823.48	2,233.51	11.27%

Annual performance

Year	Revenue	Profit	Margin	Orders
2011	484,247.50	49,543.97	10.23%	969
2012	470,532.51	61,618.60	13.10%	1,038
2013	608,473.83	81,726.93	13.43%	1,310
2014	733,947.02	93,507.51	12.74%	1,692

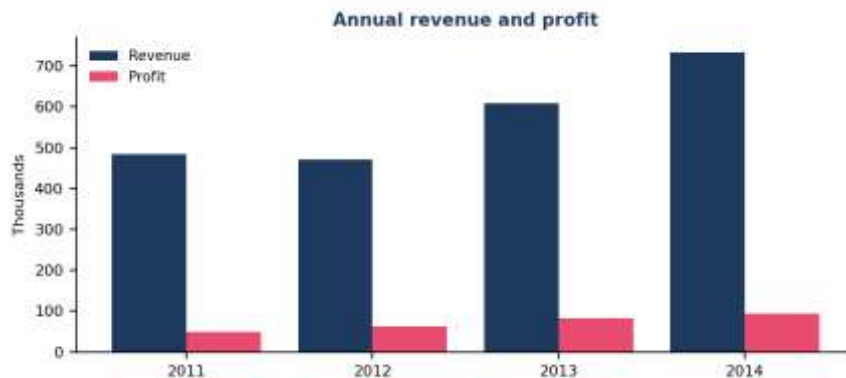


Figure 4. Annual revenue and profit (thousands)

Key findings

- **Technology leads:** highest revenue and profit, with a 17.40% margin.
- **Furniture is the profit gap:** 741,999.80 in revenue but only a 2.49% margin.
- **West leads, Central lags:** West has the highest revenue and profit; Central has the lowest margin at 7.92%.
- **Revenue is not profit:** several high-revenue products, such as the Cisco TelePresence EX90, sold at a loss.
- **Growth:** revenue rose from 470,532.51 in 2012 to 733,947.02 in 2014; peak month was November 2014 at 112,326.47.

Data quality

The dataset has zero missing values, zero exact duplicate rows and no invalid order or ship dates, which gives a reliable base for automated reporting. Monetary values carry no currency label in the dataset, so the report presents them as plain numbers.

10. Design Review and Improvements

This review is based on reading the exported workflow JSON. Items marked "Verify" depend on the live sheet and should be confirmed with a test run.

Strengths

- Clear separation of computation and interpretation, so figures come from code and not from the model.
- Only aggregates are sent to the LLM, which protects customer data and keeps prompts small.
- Alias-based column detection makes the workflow resilient to header changes.
- Null-versus-zero handling in Pipeline A prevents silent misreporting of missing metrics.
- Reports are versioned in GitHub, giving an auditable history.
- Two complementary use cases (interactive and scheduled) from one data source.

Observations and recommended improvements

#	Observation	Recommendation
1	Column detection returns the first sheet column that matches any alias. If "Product ID" appears before "Product Name" in the sheet, products would be grouped by ID. Verify the <code>detectedColumns.product</code> value in a run.	Match aliases in priority order, or use an explicit column-mapping object, and fail loudly when a key field is missing.
2	The analytics logic is duplicated across two Code nodes with different behaviour (null vs 0, rounding, currency stripping).	Move the shared logic into one sub-workflow or Code node so both pipelines always agree.
3	Dates are parsed with the JavaScript Date constructor. Ambiguous formats such as DD/MM/YYYY can be misread, and unparseable dates are skipped without being counted.	Parse with an explicit format and report the number of skipped dates in the data-quality block.
4	The report file name contains only the date. Verify whether a second run on the same day fails because the file already exists. Gmail runs after GitHub, so a GitHub error also suppresses the email.	Add a time stamp to the file name, or update the file when it exists; add an error workflow that alerts on failure.
5	The report text is written by the model, and numbers are protected only by the prompt.	Add a validation step that compares key figures in the generated text with the computed summary before publishing.
6	The chat trigger is public, and the full sheet is re-read and re-processed on every message.	Add access control, and cache the computed summary for a short period.
7	The email recipient is fixed in the node.	Move recipients to a variable or a sheet so they can be changed without editing the workflow.
8	Reports are text-only.	Embed charts or attach a PDF so the email is stakeholder-ready without opening GitHub.

Possible extensions

- Week-over-week comparison by storing each run summary.
- Alerts when a category or region margin falls below a threshold.
- Sub-category and customer-segment analysis (the dataset has 17 sub-categories and 3 segments).
- Slack or Teams delivery in addition to email.

11. Skills Demonstrated

Area	Evidence in this project
Workflow automation	Two multi-node n8n pipelines with triggers, branching context and delivery steps
JavaScript data processing	Custom KPI engine: column detection, aggregation, monthly trend, quality checks
Business analytics	Revenue, profit and margin analysis by product, category, region and time; identification of high-revenue, low-profit areas
Prompt engineering	Anti-hallucination prompts, structured report templates, observation/interpretation/recommendation separation
AI agents	Agent with memory answering questions from a computed summary
Integration	Google Sheets, GitHub and Gmail through OAuth2 credentials
Data governance	Aggregate-only AI input and a no-PII rule for public artifacts
Documentation	Version-controlled reports, exported workflow JSON and screenshots

12. Conclusion

The project shows how a data analyst can combine traditional analytics with AI to produce repeatable reporting. JavaScript produces the numbers, Claude explains them, GitHub stores them and Gmail delivers them, all without manual intervention after setup.

The sample results (2,297,200.86 revenue, 12.47% margin, a thin Furniture margin and a weak Central region) illustrate the kind of insight the pipeline surfaces every week. With a few hardening steps, such as priority-based column matching, shared analytics logic, number validation and failure alerts, the workflow would be ready for production reporting.

Workflow: Google Sheets → JavaScript KPI Analysis → Claude AI → GitHub → Gmail